

MODULE 13

Natural Language Processing

Topics Covered

Lectures 1–5 — NLP Foundations, Pipeline & Chatbot Approaches
Lectures 6–11 — Recommendation System Project (End-to-End)
Lectures 12–14 — Text Cleaning & Preprocessing
Lectures 15–17 — Text Vectorization (OHE, BoW, N-Grams, TF-IDF)
Lectures 18–20 — Word2Vec & Neural Word Embeddings

Lectures 1–5: NLP Foundations, Pipeline & Chatbot Approaches

What is NLP?

Natural Language Processing (NLP) is the branch of AI that enables computers to understand, interpret, and generate human language. It bridges the gap between unstructured text data and machine-readable information.

NLP Task	Example Use Case
Text Classification	Spam detection, sentiment analysis, topic labelling
Named Entity Recognition (NER)	Extract names, dates, locations from text
Machine Translation	Google Translate, DeepL
Question Answering	Chatbots, search engines, RAG systems
Text Summarisation	News summaries, document compression
Speech Recognition	Siri, Alexa, Google Assistant
Text Generation	ChatGPT, Copilot, email auto-complete
Sentiment Analysis	Product reviews, social media monitoring

NLP Pipeline Overview

Every NLP project follows a sequence of steps from raw text to final output:

Stage	What Happens
1. Text Acquisition	Collect raw text: CSV, web scraping, APIs, databases
2. Text Cleaning	Remove HTML, special chars, extra whitespace, noise
3. Preprocessing	Tokenise, lowercase, remove stopwords, stem/lemmatise
4. Feature Engineering	Vectorise text: BoW, TF-IDF, Word2Vec, BERT embeddings
5. Modelling	Train classifier, retrieval system, or generative model
6. Evaluation	Accuracy, F1, BLEU, ROUGE, perplexity
7. Deployment	Serve via API (Flask/FastAPI), embed in app or website

NLP Challenges

- Ambiguity — 'bank' can mean a river bank or a financial bank
- Context dependence — meaning changes with surrounding words
- Sarcasm & irony — hard for models to detect
- Multilingual text — different grammar rules, scripts, morphology
- Out-of-vocabulary words — new words, slang, domain-specific terms
- Data imbalance — far more negative reviews than positive in some datasets

Rule-Based vs Generative AI Chatbots

Aspect	Rule-Based Chatbot vs Generative AI Chatbot
How it works	Pattern matching / decision trees vs LLM generates response
Flexibility	Limited to defined rules vs Handles novel questions

Aspect	Rule-Based Chatbot vs Generative AI Chatbot
Setup	Fast to build, easy to control vs Needs LLM API or fine-tuning
Accuracy	100% on known patterns vs Probabilistic, may hallucinate
Example	FAQ bots, IVR systems vs ChatGPT, Gemini, Claude
NLP pipeline	Feature: regex/keywords vs Feature: embeddings + vector DB

□ Pipeline difference

Rule-based: Text → Clean → Match pattern → Return fixed response.

Generative AI: Text → Embed → Retrieve context (RAG) → LLM → Generate response.

Both follow the same early NLP pipeline steps — cleaning and preprocessing are universal.

Lectures 6–11: Recommendation System Project (End-to-End)

Project Overview

A content-based movie recommendation system using NLP — finds similar movies based on plot descriptions and metadata. The full pipeline: data → clean → vectorise → similarity → web app → deploy.

Phase	Tools / Technique
Data Source	TMDB 5000 Movies dataset (Kaggle)
Preprocessing	Merge DataFrames, parse JSON columns, extract keywords/cast/genres
Text Cleaning	Lowercase, remove spaces in names, combine into 'tags' column
Vectorisation	CountVectorizer (Bag of Words) — 5000 features
Similarity	Cosine similarity matrix between all movie vectors
Recommendation	For a query movie → sort by cosine similarity → return top 5
Web App	Streamlit — simple Python UI with movie poster fetch via TMDB API
Deployment	GitHub → Render (free cloud hosting)

Key Preprocessing Steps

```
import pandas as pd, ast

movies = pd.read_csv('tmdb_5000_movies.csv')
credits = pd.read_csv('tmdb_5000_credits.csv')
df = movies.merge(credits, on='title')

# Parse JSON-like string columns
def extract_names(obj):
    return [i['name'] for i in ast.literal_eval(obj)]

df['genres'] = df['genres'].apply(extract_names)
df['keywords'] = df['keywords'].apply(extract_names)

# Extract top 3 cast, director
df['cast'] = df['cast'].apply(lambda x: [i['name'] for i in ast.literal_eval(x)][0:3])

def get_director(obj):
    for i in ast.literal_eval(obj):
```

```

        if i['job'] == 'Director': return [i['name']]
    return []
df['crew'] = df['crew'].apply(get_director)

# Combine into tags (remove spaces so 'Sam Mendes' → 'SamMendes')
def clean_list(lst):
    return [i.replace(' ', '') for i in lst]

df['tags'] = (df['overview'].apply(lambda x: x.split()) +
              df['genres'].apply(clean_list) +
              df['keywords'].apply(clean_list) +
              df['cast'].apply(clean_list) +
              df['crew'].apply(clean_list))

df['tags'] = df['tags'].apply(lambda x: ' '.join(x).lower())

```

Building the Recommendation Engine

```

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Vectorise
cv = CountVectorizer(max_features=5000, stop_words='english')
vectors = cv.fit_transform(df['tags']).toarray()

# Cosine similarity matrix (4800 × 4800)
similarity = cosine_similarity(vectors)

# Recommendation function
def recommend(movie, n=5):
    idx = df[df['title'] == movie].index[0]
    scores = sorted(enumerate(similarity[idx]),
                    key=lambda x: x[1], reverse=True)[1:n+1]
    return [df.iloc[i[0]].title for i in scores]

print(recommend('Batman Begins'))
# ['The Dark Knight', 'Batman', 'The Dark Knight Rises', ...]

```

Streamlit Web App

```

# app.py
import streamlit as st
import pickle, requests

movies = pickle.load(open('movies.pkl', 'rb'))
similarity = pickle.load(open('similarity.pkl', 'rb'))

def fetch_poster(movie_id):
    url = f'https://api.themoviedb.org/3/movie/{movie_id}?api_key=YOUR_KEY'
    data = requests.get(url).json()
    return 'https://image.tmdb.org/t/p/w500' + data['poster_path']

st.title('Movie Recommendation System')
selected = st.selectbox('Choose a movie', movies['title'].values)

if st.button('Recommend'):
    names, posters = recommend(selected)
    cols = st.columns(5)
    for col, name, poster in zip(cols, names, posters):

```

```
col.text(name)
col.image(poster)
```

Deployment on Render

Step	Action
1. requirements.txt	List all dependencies: streamlit, scikit-learn, pandas, requests
2. Push to GitHub	git init → git add . → git commit → git push
3. Render	New Web Service → connect GitHub repo → set start command
4. Start command	streamlit run app.py --server.port \$PORT --server.address 0.0.0.0
5. Live URL	Render provides a public HTTPS URL — share with anyone

Lectures 12–14: Text Cleaning & Preprocessing

Basic Text Cleaning

Raw text is full of noise. Cleaning removes everything that doesn't carry meaning for your NLP task.

```
import re, string

text = '<p>Hello!! Visit https://example.com   for MORE info...</p>'

# Remove HTML tags
text = re.sub(r'<.*?>', '', text)

# Remove URLs
text = re.sub(r'http\S+|www\.\S+', '', text)

# Remove punctuation
text = text.translate(str.maketrans('', '', string.punctuation))

# Remove extra whitespace
text = re.sub(r'\s+', ' ', text).strip()

# Lowercase
text = text.lower()

print(text) # 'hello visit  for more info'
```

Advanced Text Cleaning

```
# Remove emojis
import emoji
text = emoji.replace_emoji(text, replace='')

# Remove numbers (if irrelevant)
text = re.sub(r'\d+', '', text)

# Expand contractions
import contractions
text = contractions.fix("I can't do this, it's too hard")
# → 'I cannot do this, it is too hard'

# Remove rare/frequent words (domain-specific noise)
```

```
from collections import Counter
word_freq = Counter(corpus.split())
rare_words = {w for w, c in word_freq.items() if c < 3}
text = ' '.join([w for w in text.split() if w not in rare_words])
```

Text Preprocessing

After cleaning, preprocessing transforms text into a form that algorithms can work with:

Step	Description Example
Tokenisation	Split text into tokens (words or subwords). 'I love NLP' → ['I','love','NLP']
Lowercasing	'Hello' → 'hello'. Normalises vocabulary size.
Stopword removal	Remove high-frequency words with little meaning: 'the','is','at','which'
Stemming	Chop word to root: 'running' → 'run', 'happily' → 'happili'. Fast but rough.
Lemmatisation	Reduce to dictionary base form: 'running' → 'run', 'better' → 'good'. Slower but accurate.
POS Tagging	Label each token: noun, verb, adjective — useful for NER and parsing

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

nltk.download('punkt'); nltk.download('stopwords'); nltk.download('wordnet')

text = 'The cats are running quickly through the beautiful gardens'

# Tokenise
tokens = word_tokenize(text.lower())
# ['the', 'cats', 'are', 'running', 'quickly', 'through', 'the', 'beautiful', 'gardens']

# Remove stopwords
stop = set(stopwords.words('english'))
tokens = [t for t in tokens if t not in stop and t.isalpha()]
# ['cats', 'running', 'quickly', 'beautiful', 'gardens']

# Stemming
ps = PorterStemmer()
stemmed = [ps.stem(t) for t in tokens]
# ['cat', 'run', 'quickli', 'beauti', 'garden']

# Lemmatisation
lem = WordNetLemmatizer()
lemmed = [lem.lemmatize(t, pos='v') for t in tokens]
# ['cat', 'run', 'quickly', 'beautiful', 'garden']
```

Stemming	Lemmatisation
Chops suffix by rules	Uses vocabulary + morphological analysis
Fast	Slower
May produce non-words ('happili')	Always produces real words
Good for search engines, BoW	Better for chatbots, Q&A, generation tasks

Lectures 15–17: Text Vectorization

ML models require numbers. Vectorisation converts text into numerical representations that capture meaning and can be fed into algorithms.

One-Hot Encoding (OHE) for Text

Each word in the vocabulary gets its own binary column. A document is represented as a vector of 0s and 1s.

```
# Example vocabulary: ['cat', 'dog', 'fish', 'bird']
# 'I saw a cat' → [1, 0, 0, 0]
# 'I saw a dog' → [0, 1, 0, 0]
```

```
from sklearn.preprocessing import LabelBinarizer

words = ['cat', 'dog', 'fish', 'bird']
lb = LabelBinarizer()
ohe = lb.fit_transform(words)
print(ohe)
```

❑ Limitations of OHE for text

Vocabulary size = vector size → massive sparse vectors for large corpora.

No semantic relationship — 'cat' and 'dog' are as different as 'cat' and 'galaxy'.

Does not capture word frequency or document context.

Bag of Words (BoW)

Counts word frequencies in each document. Discards word order but captures how often each word appears.

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    'I love NLP and I love Python',
    'NLP is the future of AI',
    'Python is great for NLP',
]

cv = CountVectorizer()
X = cv.fit_transform(corpus)

print(cv.get_feature_names_out())
# ['ai', 'and', 'for', 'future', 'great', 'is', 'love', 'nlp', 'of', 'python', 'the']
print(X.toarray())
# [[0, 1, 0, 0, 0, 0, 2, 1, 0, 1, 0],
#  [1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1],
#  [0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0]]

# Limit vocabulary size
cv = CountVectorizer(max_features=1000, stop_words='english')
```

N-Grams

Instead of single words (unigrams), N-grams capture sequences of N consecutive words — preserving some local context that BoW loses.

```
# Unigram: 'I love NLP' → ['I', 'love', 'NLP']
```

```
# Bigram: 'I love NLP' → ['I love', 'love NLP']
# Trigram: 'I love NLP' → ['I love NLP']

cv_bigram = CountVectorizer(ngram_range=(1, 2)) # unigrams + bigrams
cv_trigram = CountVectorizer(ngram_range=(1, 3))

X = cv_bigram.fit_transform(corpus)
print(cv_bigram.get_feature_names_out())
# includes: 'love nlp', 'nlp python', 'nlp future', etc.
```

□ Choosing N

Unigrams (1,1): simplest, lowest dimensionality, loses context.

Bigrams (1,2): captures phrases, good balance. Most common in practice.

Trigrams (1,3): better context but sparse — needs large corpus.

TF-IDF (Term Frequency – Inverse Document Frequency)

TF-IDF weights words by how frequently they appear in a document relative to how common they are across all documents. Rare but important words get high scores.

Component	Formula Meaning
TF (Term Frequency)	$TF(t,d) = \text{count}(t \text{ in } d) / \text{total words in } d$ — how important is word t in doc d ?
IDF (Inverse Doc Freq)	$IDF(t) = \log(N / df(t))$ — penalises words common across ALL documents (like 'the')
TF-IDF	$TF-IDF(t,d) = TF(t,d) \times IDF(t)$ — high score = word is frequent in this doc but rare overall

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=5000, stop_words='english',
                        ngram_range=(1, 2))
X = tfidf.fit_transform(corpus)

print(X.shape) # (3, n_features)
print(X.toarray().round(3))

# Get top TF-IDF words for a document
import pandas as pd
feature_names = tfidf.get_feature_names_out()
doc_tfidf = pd.Series(X[0].toarray()[0], index=feature_names)
print(doc_tfidf.sort_values(ascending=False).head(10))
```

Method	Captures Best For
One-Hot Encoding	Presence only → tiny vocabularies, baseline
Bag of Words	Word frequency → topic classification, recommendation
N-Grams	Word sequences → phrase detection, sentiment
TF-IDF	Relative importance → search engines, document similarity

Lectures 18–20: Word2Vec & Neural Word Embeddings

Why Word Embeddings?

BoW and TF-IDF treat every word as independent — 'king' and 'queen' are as unrelated as 'king' and 'banana'. Word embeddings map words to dense vectors where similar words are close in vector space.

□ The key insight

king - man + woman $\approx$ queen

Paris - France + Italy $\approx$ Rome

Word embeddings capture semantic relationships — meaning is encoded in geometry.

Word2Vec Overview

Word2Vec (Google, 2013) trains a shallow neural network to predict words from context. The hidden layer weights become the word embeddings — dense vectors of typically 100–300 dimensions.

Aspect	Details
Output	Each word → a dense float vector of size d (e.g. 100, 200, 300)
Key property	Semantically similar words have high cosine similarity
Training data	Any large text corpus — Wikipedia, news, your own data
Pre-trained	Google News (3M words, 300d), GloVe (Wikipedia), FastText
Library	gensim is the standard Python library for Word2Vec

Two Word2Vec Architectures

Architecture	How It Works When to Use
CBOW (Continuous Bag of Words)	Predict the TARGET word from surrounding context words. Faster, works well for frequent words.
Skip-Gram	Predict CONTEXT words from the target word. Slower but better for rare words and large datasets.

```
# CBOW example:
# Context: ['The', '___', 'sat', 'on', 'the', 'mat']
# Predict: 'cat'

# Skip-Gram example:
# Input: 'cat'
# Predict: ['The', 'sat', 'on', 'the', 'mat'] (within window)
```

Training Word2Vec with Gensim

```
from gensim.models import Word2Vec
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize

# Prepare corpus - list of tokenised sentences
text = open('my_corpus.txt', 'r').read()
sentences = [word_tokenize(s.lower())
              for s in sent_tokenize(text)]

# Train model
model = Word2Vec(
    sentences      = sentences,
    vector_size   = 100,      # embedding dimensions
    window        = 5,       # context window size
    min_count     = 2,       # ignore words appearing < 2 times
    workers       = 4,       # parallel threads
```

```

    sg          = 0,          # 0=CBOW, 1=Skip-Gram
    epochs      = 10

)

# Save and load
model.save('word2vec.model')
model = Word2Vec.load('word2vec.model')

```

Using a Trained Word2Vec Model

```

wv = model.wv    # word vectors accessor

# Get vector for a word
vector = wv['python']          # shape: (100,)
print(vector[:5])              # [-0.23, 0.41, ...]

# Most similar words
print(wv.most_similar('python', topn=5))
# [('java', 0.89), ('ruby', 0.87), ('code', 0.84), ...]

# Cosine similarity between two words
print(wv.similarity('king', 'queen'))    # e.g. 0.73

# Word analogy: king - man + woman
result = wv.most_similar(positive=['king', 'woman'],
                        negative=['man'], topn=1)
print(result)    # [('queen', 0.81)]

# Words that don't fit
print(wv.doesnt_match(['python', 'java', 'banana', 'ruby']))
# 'banana'

```

Document Vectors from Word2Vec

Word2Vec gives word-level vectors. To represent a full document, average the vectors of all its words:

```

import numpy as np

def document_vector(text, model):
    words = word_tokenize(text.lower())
    vectors = [model.wv[w] for w in words if w in model.wv]
    if not vectors:
        return np.zeros(model.vector_size)
    return np.mean(vectors, axis=0)

# Use as features for ML
X = np.array([document_vector(doc, model) for doc in df['text']])
# X shape: (n_documents, 100)

from sklearn.linear_model import LogisticRegression
clf = LogisticRegression()
clf.fit(X_train, y_train)

```

Beyond Word2Vec — Modern Embeddings

Model	Key Improvement
GloVe (2014)	Global co-occurrence statistics — better on analogies

Model	Key Improvement
FastText (2016)	Subword embeddings — handles OOV words by using character n-grams
ELMo (2018)	Context-dependent — same word gets different vector based on sentence
BERT (2018)	Bidirectional transformer — rich contextual embeddings, state-of-the-art
Sentence-BERT (2019)	Optimised for sentence similarity — used in RAG/semantic search
<pre># Using pre-trained FastText (handles misspellings & rare words) import fasttext.util fasttext.util.download_model('en', if_exists='ignore') ft = fasttext.load_model('cc.en.300.bin') vec = ft.get_word_vector('artificialintelligence') # works even for OOV! print(vec.shape) # (300,)</pre>	

End of Module 13 Notes — Generative AI Course